

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Industry Problem-Based Assignment
COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA010**
1

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A
Industry Problem-Based Assignment

Code of Conduct:

I Shaik Mubarak(192425194) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	Healthcare	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	Banking	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	Cybersecurity	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	Manufacturing	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	Agriculture	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	Telecommunications	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	E-Commerce	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.

9	Power & Energy	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	Human Resources	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference traces.	7–8% – Provides relevant test cases with mostly accurate results.	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				

INDUSTRY PROBLEM-BASED ASSIGNMENT

INTELLIGENT CYBER THREAT DETECTION AND RESPONSE ADVISORY SYSTEM USING PROLOG

1.2 Abstract

Cybersecurity systems continuously generate large amounts of security events such as failed login attempts, unusual login locations, privilege changes, suspicious file access, and abnormal network traffic. Identifying potentially harmful activities from these events is an important task for Security Operations Centers. This project presents a rule-based Cyber Threat Detection and Response Advisory System developed using SWI-Prolog. The system represents security events as Prolog facts and uses production rules to identify possible threats such as brute-force attacks, unauthorized access, privilege escalation, malware activity, and network intrusion.

The system demonstrates important Artificial Intelligence and Prolog concepts including knowledge representation, production rules, forward chaining, backward chaining, unification, backtracking, and declarative programming. The system also provides recommended security actions and explanations for the detected threats. Multiple cybersecurity scenarios are used to test the system's reasoning accuracy, consistency, and rule coverage. The proposed system demonstrates how a rule-based expert system can provide explainable preliminary threat analysis in a cybersecurity environment.

1.3 Introduction

Cybersecurity is an important area for organizations because unauthorized access and suspicious activities can cause data loss, service disruption, and security breaches. Security Operations Centers monitor different events generated by users, devices, applications, and networks. Some individual events may not clearly indicate an attack. However, when multiple related events occur together, they can indicate a possible security threat. For example, repeated failed login attempts combined with an unusual login location may indicate a brute-force or unauthorized-access attempt.

The proposed system uses Prolog to represent these security events as facts and applies production rules to infer possible threats. The assessment specifically requires an expert system that identifies possible cybersecurity threats and recommends appropriate actions.

Prolog is suitable for this application because cybersecurity knowledge can be naturally represented using logical facts and IF-THEN-style rules.

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

A Security Operations Center receives large numbers of security events from users, computers, applications, and networks. Manually analyzing every event and determining whether multiple events indicate a security threat can be time-consuming.

The problem addressed in this project is to develop a Prolog-based expert system that analyzes security events, identifies probable cybersecurity threats, and recommends suitable response actions.

The system uses predefined rules to connect observed events with possible threats.

Inputs

The system considers events such as:

- Repeated failed logins
- Unusual login location
- Privilege escalation
- Suspicious file access
- Abnormal network traffic

- Unauthorized access attempts

Expected Outputs

The system produces:

- Possible security threat
- Reason for the detected threat
- Recommended response action

Stakeholders

1. Security Operations Center analysts
2. System administrators
3. Network security teams
4. IT organizations
5. Security managers

Constraints

The proposed system is an academic prototype. It uses a predefined knowledge base and simplified security events. It does not replace professional security monitoring systems or complete incident-response procedures.

2.2 Objectives

The main objectives are:

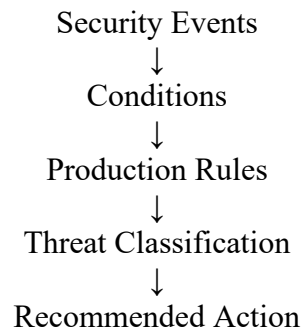
1. To develop a rule-based cybersecurity expert system using SWI-Prolog.
2. To represent security events using Prolog facts and predicates.
3. To develop production rules for identifying possible cyber threats.
4. To implement forward chaining for event-driven threat detection.
5. To demonstrate backward chaining for verifying a suspected threat.
6. To demonstrate unification and backtracking in Prolog.
7. To recommend suitable response actions for detected threats.
8. To provide explanations for the generated threat classification.
9. To test the system using at least five cybersecurity scenarios.
10. To evaluate reasoning accuracy, consistency, rule coverage, and practical usefulness.

3. DOMAIN ANALYSIS & KNOWLEDGE ACQUISITION

3.1 Domain Analysis

The selected domain is **cybersecurity threat detection**.

The system analyzes relationships between:



3.2 Domain Entities

Entity	Examples
User	user1, user2
Security Event	Failed login
Location	Unknown location
Access	Suspicious file access
Network Activity	Abnormal traffic

Privilege	Unauthorized privilege change
Threat	Brute-force attack
Response	Block account

3.3 Security Events

The main events represented in the system are:

- failed_logins
- unusual_location
- privilege_change
- suspicious_file_access
- abnormal_traffic
- unauthorized_access

3.4 Possible Threats

Security Conditions	Possible Threat
Failed logins + unusual location	Brute-force attack
Unusual location + unauthorized access	Unauthorized access
Privilege change + suspicious file access	Privilege escalation
Suspicious file access + abnormal traffic	Possible malware activity
Unauthorized access + abnormal traffic	Network intrusion

3.5 Knowledge Acquisition

Cybersecurity knowledge was obtained by analyzing common relationships between security events and potential threat categories. Standard cybersecurity concepts such as authentication anomalies, unauthorized access, privilege misuse, suspicious file activity, and abnormal network behavior were considered while developing the simplified knowledge base.

The knowledge acquisition process involved:

1. Identifying important cybersecurity events.
2. Identifying possible threat categories.
3. Establishing relationships between events and threats.
4. Converting these relationships into IF–THEN production rules.
5. Representing the rules as Prolog predicates.

The knowledge base is intentionally simplified so that the reasoning process can be clearly demonstrated.

4. KNOWLEDGE REPRESENTATION & PRODUCTION RULES

This section carries **15%**, so it should be one of the strongest parts of the report. The rubric expects a complete, consistent and well-structured Prolog knowledge base containing facts, predicates and production rules.

4.1 Prolog Facts

Facts represent known security observations.

Example:

failed_logins(user1).

unusual_location(user1).

privilege_change(user2).

suspicious_file_access(user2).

suspicious_file_access(user3).

abnormal_traffic(user3).

unauthorized_access(user4).

abnormal_traffic(user4).

A fact such as:

failed_logins(user1).

means that repeated login failures have been observed for user1.

4.2 Production Rules

Rule 1 – Brute-Force Attack

threat(User, brute_force_attack) :-

failed_logins(User),

unusual_location(User).

IF repeated failed logins occur **AND** the login originates from an unusual location
THEN there is a possible brute-force attack.

Rule 2 – Unauthorized Access

threat(User, unauthorized_access) :-

unusual_location(User),

unauthorized_access(User).

IF an unusual login location **AND** unauthorized access are detected
THEN classify the activity as possible unauthorized access.

Rule 3 – Privilege Escalation

threat(User, privilege_escalation) :-

privilege_change(User),

suspicious_file_access(User).

Rule 4 – Malware Activity

threat(User, possible_malware) :-

suspicious_file_access(User),

abnormal_traffic(User).

Rule 5 – Network Intrusion

threat(User, network_intrusion) :-

unauthorized_access(User),

abnormal_traffic(User).

4.3 Response Rules

The system also provides recommendations.

response(brute_force_attack,

'Block suspicious login attempts and review account activity.').

response(unauthorized_access,

'Disable suspicious access and verify the user identity.').

response(privilege_escalation,

'Review privilege changes and remove unauthorized privileges.').

response(possible_malware,

'Isolate the affected system and perform malware analysis.').

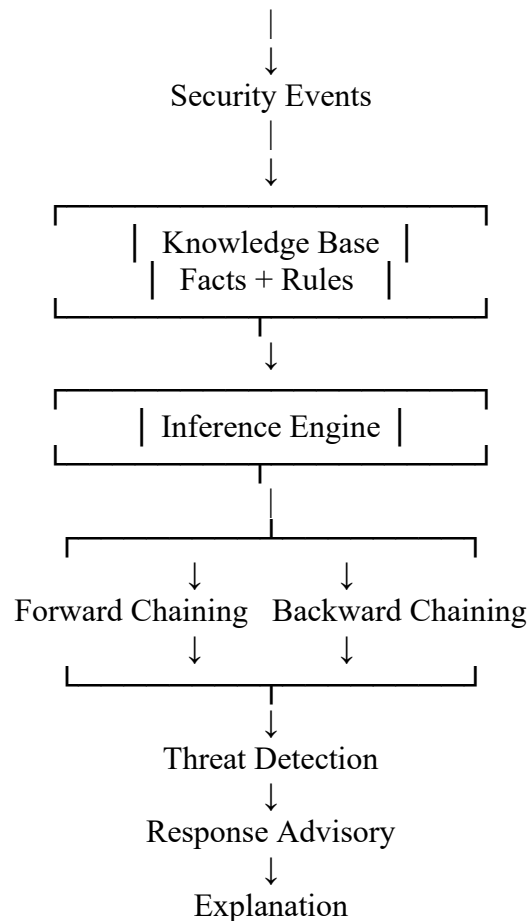
response(network_intrusion,

'Investigate network traffic and isolate the affected system.').

This allows the system to provide not only a threat classification but also a suitable response recommendation.

5. EXPERT SYSTEM ARCHITECTURE

The proposed system consists of the following major components:



5.1 Components

1. User Interface

The user or security analyst provides security events or selects a user/security case.

2. Knowledge Base

Contains cybersecurity facts and production rules.

3. Inference Engine

Applies logical reasoning to determine whether a threat can be derived.

4. Threat Classification Module

Identifies the probable security threat.

5. Response Advisory Module

Provides a recommended security action.

6. Explanation Facility

Explains which observed events caused the threat classification.

6. INFERENCE MECHANISM & ALGORITHMS

The assessment requires demonstration of forward chaining, backward chaining, unification and backtracking. These are therefore explicitly included in the system.

6.1 Forward Chaining

Forward chaining is a data-driven reasoning approach.

It starts with known security events and applies rules to derive threats.

Example

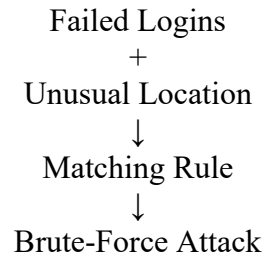
Known facts:

failed_logins(user1)
unusual_location(user1)

Rule:

IF failed_logins AND unusual_location
THEN brute_force_attack

Reasoning:



Algorithm

1. Start with known security events.
2. Check all production rules.
3. Find rules whose conditions are satisfied.
4. Fire the matching rules.
5. Generate the corresponding threat.
6. Generate the recommended response.
7. Continue until no new conclusion is produced.

Query

?- forward_chain(user1).

Expected Result

Derived Threat: brute_force_attack

6.2 Backward Chaining

Backward chaining is a **goal-driven reasoning approach**.

It begins with a suspected threat and attempts to prove it from available facts.

For example:

?- threat(user1, brute_force_attack).

Prolog searches for a rule that can prove the goal:

threat(User, brute_force_attack) :-

failed_logins(User),
unusual_location(User).

It then checks:

failed_logins(user1) → TRUE

unusual_location(user1) → TRUE

Therefore:

brute_force_attack(user1) → TRUE

Algorithm

1. Select a suspected threat as the goal.
2. Find a rule that can produce the goal.
3. Identify the conditions required by the rule.
4. Check the conditions against known facts.
5. If all conditions are satisfied, prove the goal.
6. If a condition fails, backtrack and try another rule.

6.3 Unification

Unification is the process of matching Prolog terms and variables.

Consider:
threat(User, brute_force_attack).
and:
?- threat(user1, brute_force_attack).
Prolog matches:
User = user1
This is called **unification**.
The same rule can therefore be applied to different users without writing a separate rule for every user.

6.4 Backtracking

Backtracking allows Prolog to search for alternative solutions.

Suppose a user has:

suspicious_file_access

abnormal_traffic

unauthorized_access

The system may find:

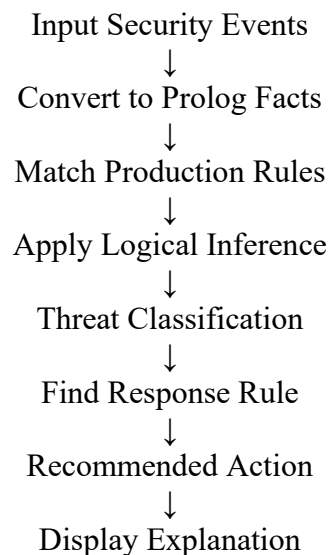
possible_malware

network_intrusion

If Prolog finds one solution, it can backtrack and search for another applicable rule.

This feature is useful when multiple threats may exist for the same security event set.

6.5 Overall Inference Process



7. PROLOG IMPLEMENTATION

The system is implemented using **SWI-Prolog**.

The implementation uses facts, rules, variables, predicates, unification and backtracking.

7.1 Important Predicates

Predicate	Purpose
failed_logins/1	Represents repeated login failures
unusual_location/1	Represents unusual login location
privilege_change/1	Represents privilege changes
suspicious_file_access/1	Represents suspicious file activity
abnormal_traffic/1	Represents abnormal network activity
unauthorized_access/1	Represents unauthorized access

threat/2	Determines possible cybersecurity threat
response/2	Provides recommended action
forward_chain/1	Demonstrates forward reasoning
backward_diagnosis/2	Demonstrates goal-driven reasoning
diagnose/1	Displays complete threat diagnosis
explain/2	Explains the reasoning

7.2 Example Prolog Rule

```
threat(User, brute_force_attack) :-
    failed_logins(User),
    unusual_location(User).
```

7.3 Backward Query

```
?- threat(user1, brute_force_attack).
```

Expected:

true.

7.4 Forward Query

```
?- forward_chain(user1).
```

Expected:

Derived Threat: brute_force_attack

7.5 Complete Diagnosis

```
?- diagnose(user1).
```

Expected:

Possible Threat: brute_force_attack

Recommended Action:

Block suspicious login attempts and review account activity.

The implementation is designed around the rubric's requirement for effective use of facts, rules, predicates, unification, backtracking and inference.

8. TEST CASES & RESULTS

The assessment requires a minimum of five industry-based test cases, including queries, expected outputs, actual outputs and inference traces.

Important: In the final report, the "Actual Output" column should be filled using the output you obtain from your SWI-Prolog program. We should not claim an output was actually obtained until we run the code.

Test Case 1 – Brute-Force Attack

Input

Repeated failed logins

Unusual login location

Query

```
?- threat(user1, brute_force_attack).
```

Expected Output

true.

Inference Trace

```
failed_logins(user1)
```

+

```
unusual_location(user1)
```

↓

```
brute_force_attack
```

Recommended Action

Block suspicious login attempts and review account activity.

Test Case 2 – Privilege Escalation

Input

Privilege change
Suspicious file access

Query

?- threat(user2, privilege_escalation).

Expected Output

true.

Inference Trace

privilege_change(user2)
+
suspicious_file_access(user2)
↓
privilege_escalation

Recommended Action

Review privilege changes and remove unauthorized privileges.

Test Case 3 – Possible Malware Activity

Input

Suspicious file access
Abnormal network traffic

Query

?- threat(user3, possible_malware).

Expected Output

true.

Inference Trace

suspicious_file_access(user3)
+
abnormal_traffic(user3)
↓
possible_malware

Recommended Action

Isolate the affected system and perform malware analysis.

Test Case 4 – Network Intrusion

Input

Unauthorized access
Abnormal network traffic

Query

?- threat(user4, network_intrusion).

Expected Output

true.

Inference Trace

unauthorized_access(user4)
+
abnormal_traffic(user4)
↓
network_intrusion

Recommended Action

Investigate network traffic and isolate the affected system.

Test Case 5 – Unauthorized Access

Input

Unusual login location

Unauthorized access

Query

?- threat(user5, unauthorized_access).

Expected Output

true.

Inference Trace

unusual_location(user5)

+

unauthorized_access(user5)

↓

unauthorized_access

Recommended Action

Disable suspicious access and verify the user's identity.

Test Case 6 – No Matching Threat

Input

Normal login

Normal network traffic

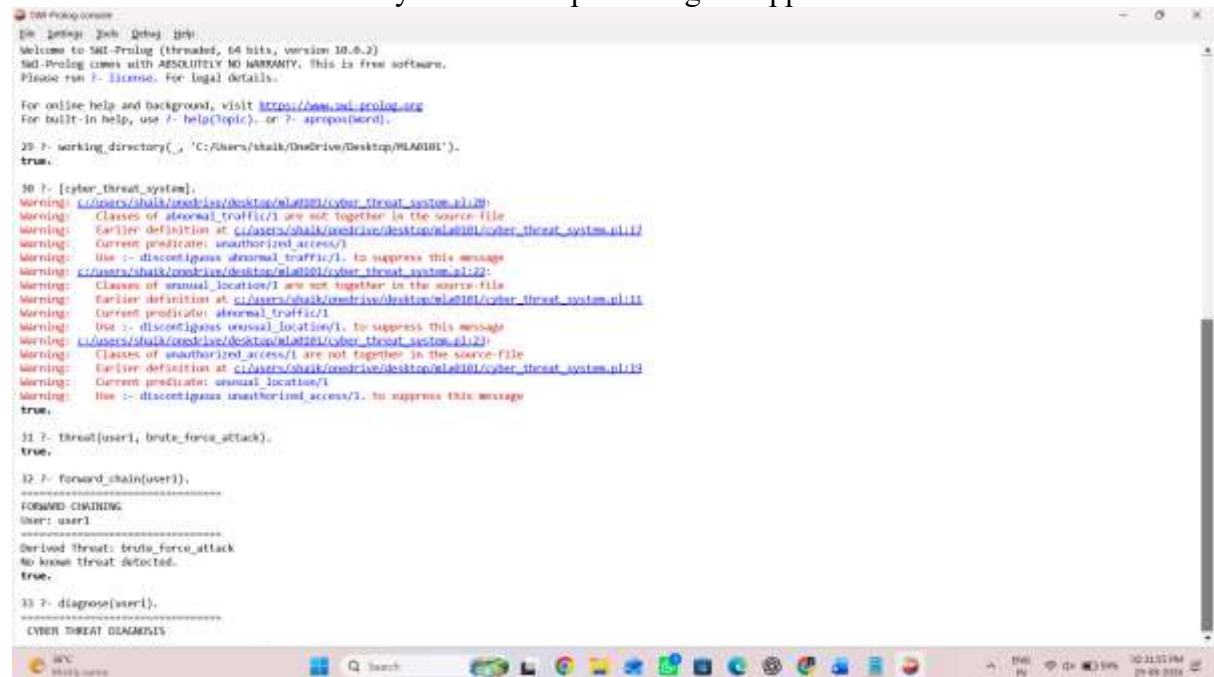
Query

?- diagnose(user6).

Expected Output

No known threat detected.

This test checks whether the system avoids producing unsupported conclusions.



```
SWI-Prolog console
[0] ?- help. [ok] [help] [help]
Welcome to SWI-Prolog (threaded, 64 bits, version 10.6.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. For legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(topic). or ?- apropos(word).

20 ?- working_directory(_, 'C:/Users/shaik/OneDrive/Desktop/PLA0101').
true.

30 ?- [cyber_threat_system].
Warning: c:/users/shaik/onedrive/desktop/pla0101/cyber_threat_system.pl:20:
Warning: Classes of abnormal_traffic/1 are not together in the source file
Warning: Earlier definition at c:/users/shaik/onedrive/desktop/pla0101/cyber_threat_system.pl:11
Warning: Current predicate: unauthorized_access/1
Warning: Use :- discontiguous abnormal_traffic/1. to suppress this message
Warning: c:/users/shaik/onedrive/desktop/pla0101/cyber_threat_system.pl:22:
Warning: Classes of unusual_location/1 are not together in the source file
Warning: Earlier definition at c:/users/shaik/onedrive/desktop/pla0101/cyber_threat_system.pl:11
Warning: Current predicate: abnormal_traffic/1
Warning: Use :- discontiguous unusual_location/1. to suppress this message
Warning: c:/users/shaik/onedrive/desktop/pla0101/cyber_threat_system.pl:23:
Warning: Classes of unauthorized_access/1 are not together in the source file
Warning: Earlier definition at c:/users/shaik/onedrive/desktop/pla0101/cyber_threat_system.pl:11
Warning: Current predicate: unusual_location/1
Warning: Use :- discontiguous unauthorized_access/1. to suppress this message
true.

31 ?- threat(user1, brute_force_attack).
true.

32 ?- forward_chain(user1).
=====
FORWARD CHAINING
User: user1
=====
Derived threat: brute_force_attack
No known threat detected.
true.

33 ?- diagnose(user1).
=====
CYBER THREAT DIAGNOSIS
=====
```

```

SWI-Prolog console
File Edit Shell Debug Help
CYBER THREAT DIAGNOSIS
=====
User: user2
Possible Threat: privilege_escalation
Recommended Action: Review privilege changes and remove unauthorized privileges.
true.

15 ?- diagnose(user2).
=====
CYBER THREAT DIAGNOSIS
=====
User: user3
Possible Threat: possible_malware
Recommended Action: Isolate the system and perform malware analysis.
true.

16 ?- diagnose(user4).
=====
CYBER THREAT DIAGNOSIS
=====
User: user4
Possible Threat: network_intrusion
Recommended Action: Investigate network traffic and isolate the affected system.
true.

17 ?- explain(user1, brute_force_attack).
Reason: failed login + unusual location.
true.

18 ?- explain(user3, possible_malware).
Reason: suspicious file access + abnormal traffic.
true.

19 ?- threat(user3, X).
X = possible_malware .

40 ?-

```

Test Results Table

Test Case	Observed Events	Expected Threat	Actual Output
TC1	Failed logins + unusual location	Brute-force attack	To be recorded from SWI-Prolog
TC2	Privilege change + suspicious file access	Privilege escalation	To be recorded
TC3	Suspicious file access + abnormal traffic	Possible malware	To be recorded
TC4	Unauthorized access + abnormal traffic	Network intrusion	To be recorded
TC5	Unusual location + unauthorized access	Unauthorized access	To be recorded
TC6	No suspicious conditions	No known threat	To be recorded

9. ANALYSIS, LIMITATIONS & FUTURE ENHANCEMENTS

9.1 Analysis of Results

The proposed expert system demonstrates that cybersecurity events can be represented using logical facts and rules. When the conditions of a production rule are satisfied, Prolog can derive the corresponding threat.

Reasoning Accuracy

The system produces a threat when all conditions required by the corresponding rule are satisfied. This makes the reasoning predictable and explainable.

Consistency

The same set of security facts produces the same logical conclusion because the rules are predefined.

Rule Coverage

The current system covers several important threat categories:

- Brute-force attack
- Unauthorized access

- Privilege escalation
- Possible malware activity
- Network intrusion

Practical Usefulness

The system can assist security analysts by providing a preliminary classification of suspicious events and recommending an appropriate action.

Explainability

Unlike a black-box classification system, the rule-based system can explain its conclusion by identifying the facts that satisfied a particular rule.

The rubric specifically expects analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.

9.2 Limitations

The system has the following limitations:

1. It uses a predefined cybersecurity knowledge base.
2. It handles simplified security events rather than complete real-world logs.
3. It does not perform real-time network monitoring.
4. It does not directly connect to SIEM or IDS systems.
5. Multiple complex attacks may require additional rules.
6. The system does not calculate a numerical risk score.
7. The recommendations are general and should be verified by security professionals.
8. The system cannot automatically respond to attacks in the current implementation.

9.3 Future Enhancements

The system can be enhanced by:

1. Integrating real-time SIEM data.
2. Connecting the system with intrusion detection systems.
3. Adding more cybersecurity attack patterns.
4. Adding risk and severity levels.
5. Supporting real-time event streams.
6. Developing a graphical user interface.
7. Adding automatic alert generation.
8. Integrating threat-intelligence information.
9. Maintaining historical security events.
10. Adding automated response mechanisms for selected threats.

10. CONCLUSION & REFERENCES

10.1 Conclusion

The Intelligent Cyber Threat Detection and Response Advisory System Using Prolog demonstrates how a rule-based expert system can be applied to a real-world cybersecurity problem.

The system represents security events as Prolog facts and uses production rules to identify possible threats. It demonstrates both forward and backward chaining, allowing the system to perform data-driven threat detection as well as goal-driven threat verification. Important Prolog concepts such as unification, backtracking, logical inference, variables, predicates, and declarative programming are demonstrated through the implementation.

The testing methodology covers multiple cybersecurity scenarios including brute-force attacks, unauthorized access, privilege escalation, possible malware activity, and network intrusion. The system also provides recommended response actions and explanations. Although

the current system is a simplified academic prototype, it demonstrates the usefulness of Prolog for developing explainable, rule-based cybersecurity decision-support systems.

10.2 References

You can use the following references in the report:

1. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020.
2. I. Bratko, Prolog Programming for Artificial Intelligence, 4th ed., Pearson, 2011.
3. SWI-Prolog Developers, SWI-Prolog Reference Manual, SWI-Prolog Documentation.
4. National Institute of Standards and Technology, Computer Security Incident Handling Guide, NIST Special Publication 800-61.
5. MITRE, MITRE ATT&CK: Enterprise Knowledge Base, MITRE Corporation.
6. W. Stallings, Effective Cybersecurity: A Guide to Using Best Practices and Standards, Pearson.